

Relationships

Contents

- Eloquent Relationships
 - *one-to-one*
 - *one-to-many*
 - *many-to-many*
- Querying Relations
- Example
- References

Eloquent Relationships

Eloquent Model Relationship Defining

- We can define model relationships by creating methods within model class that return the type of relationships by using **relationship method** and the name of method will be the relationship name, e.g.:

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class Product extends Model
{
    function category(): BelongsTo {
        return $this->belongsTo(Category::class);
    }
}
```

- We will learn about relationship methods in the next several slides.
- The relationship methods, e.g., `belongsTo()`, are available to our model via the model's `Illuminate\Database\Eloquent\Model` base class.

Eloquent Model Relationship Defining (cont.)

- When we have defined relationships, we can get the related model by calling them like **object properties** accessing, e.g.

```
$product = Product::where('code', 'PD001')->firstOrFail();
```

```
/*
```

```
 * With the following code Eloquent will generate SQL  
 * related to method name that declares the relationship  
 * then send SQL query to get category data.
```

```
*/
```

```
$category = $product->category; // property accessing
```

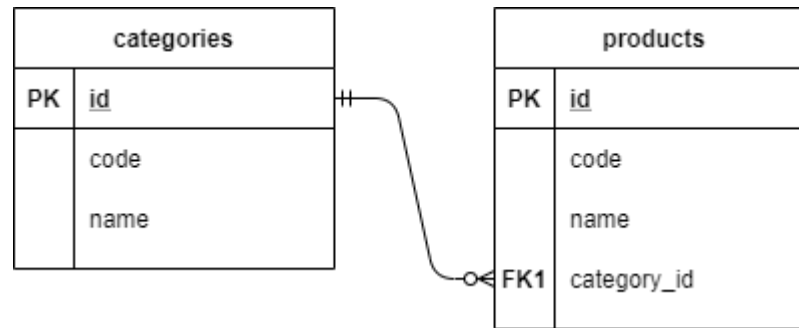
- Keep in your mind that we **declare them by using methods** but **retrieve their data by using properties** accessing.

Eloquent Model Relationship Declaration (cont.)

- The relationships defined in model class **do not force** our database to have the same relationships as model.
 - **They are just used to generate SQL.**
- But for many reasons, e.g.:
 - performance
 - data integrity

we also define our database to have the same relationships as defined in model.

one-to-many relationship



- When we want to define *one-to-many* relationship, we put `hasMany()` relationship on `Category` model side and put `belongsTo()` relationship on `Product` model side.
 - Remember that we put `belongsTo()` relationship on the side that **contains foreign key**.
- In the case we do not want to get the list of products on `Category` model side, called *unidirectional*, we just do not add relation method on it.

one-to-many relationship (cont.)

class Category

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasMany;

class Category extends Model
{
    function products(): HasMany {
        return $this->hasMany(Product::class);
    }
}
```

class Product

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class Product extends Model
{
    function category(): BelongsTo {
        return $this->belongsTo(Category::class);
    }
}
```

Naming convention — belongsTo ()

- Because of the belongsTo () relationship requires foreign key so the name of foreign key in table uses the following convention.
 - Eloquent determines the foreign key name by examining **the name of the relationship** and suffixing the method name with **_id**.
 - So, in this case, Eloquent assumes that the Product model has a category_id column.
 - category comes from the name of method, or category () .

Naming convention — `hasMany()`

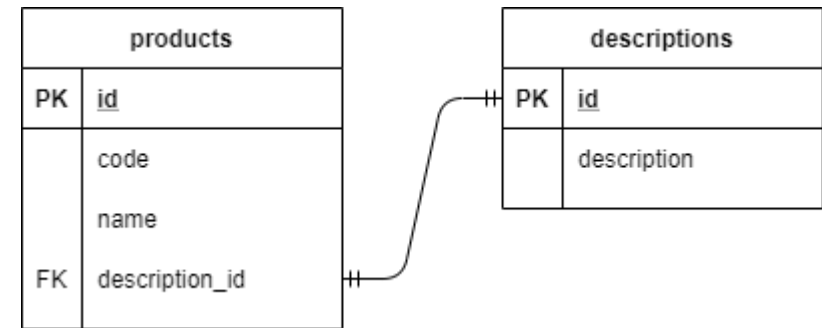
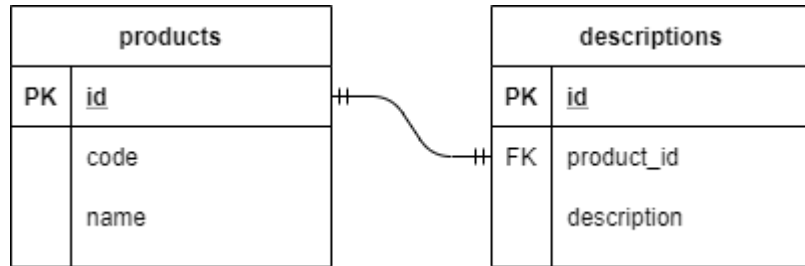
- Because of the `hasMany()` relationship requires foreign key on the other side of the relationship so the referred foreign key in the other table uses the following convention.
 - Eloquent will take the **"snake_case" name of the parent model** and suffix it with **`_id`**.
 - So, in this example, Eloquent will assume the **foreign key column** on the **Product** model for `Category` model side is **`category_id`**.
 - `category` comes from the class name of `Category` model.

Naming convention — specific foreign key

- From the naming convention of `belongsTo()` and `hasMany()`, they may produce the **difference foreign key name**.
 - So, in this case, it **forces us to name** the relationship on `belongsTo()` side to `category()`.
- In some cases, we cannot use the previous naming convention, so Eloquent allows us to specify the name of foreign key by passing the **second argument** to relationship method, e.g.

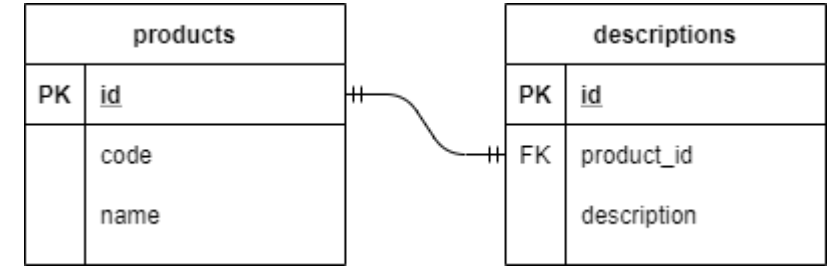
```
return $this->hasMany(Product::class, 'category_id');
```

one-to-one relationship



- When we want to define *one-to-one* relationship, we have to add foreign key on either side of models.
 - The side that has **foreign key** uses **belongsTo()** relationship method.
 - And the other side uses **hasOne()** relationship method.
- And same as *one-to-many* relationship, for *unidirectional*, we can ignore the relationship on **hasOne()** side.
- The naming convention of **hasOne()** is same as **hasMany()**.

one-to-one relationship foreign key on descriptions



class Product

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\HasOne;

class Product extends Model
{
    function description(): HasOne {
        return $this->hasOne(Description::class);
    }
}
```

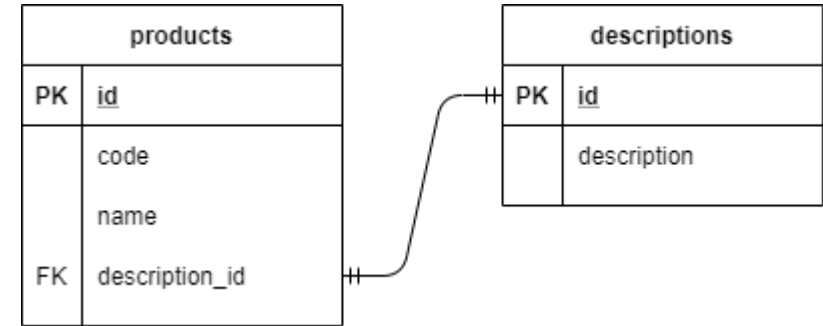
class Description

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsTo;

class Description extends Model
{
    function product(): BelongsTo {
        return $this->belongsTo(Product::class);
    }
}
```

one-to-one relationship foreign key on products



class Product

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

use
Illuminate\Database\Eloquent\Relations\BelongsTo;

class Product extends Model
{
    function description(): BelongsTo {
        return $this->belongsTo(Description::class);
    }
}
```

class Description

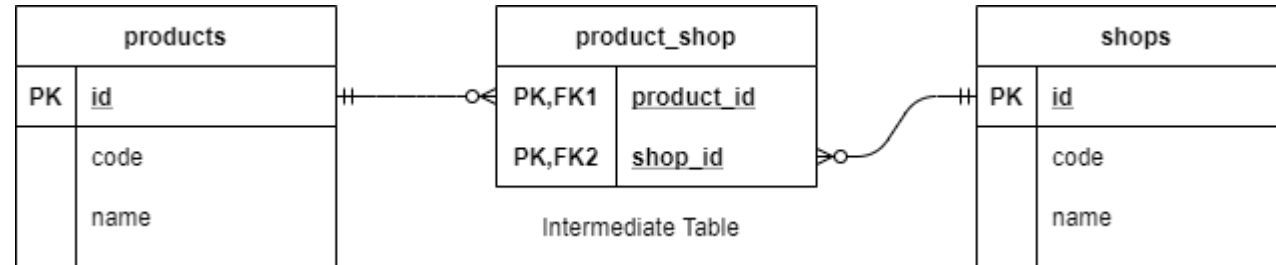
```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;

use Illuminate\Database\Eloquent\Relations\HasOne;

class Description extends Model
{
    function product(): HasOne {
        return $this->hasOne(Product::class);
    }
}
```

many-to-many relationship



- For *many-to-many* relationship, we require the **intermediate table** to link two models together.
 - On the both side of models use `belongsToMany()` relationship method.
- By default, intermediate table:
 - use `product_id` and `shop_id` to be **composite primary key**.
 - can has **`created_at`** and **`updated_at`** timestamp by using **`withTimestamps()`** when we define relationship.

many-to-many relationship

class Product

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsToMany;

class Product extends Model
{
    function shops(): BelongsToMany {
        return $this
            ->belongsToMany(Shop::class)
            ->withTimestamps();
    }
}
```

class Shop

```
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Relations\BelongsToMany;

class Shop extends Model
{
    function products(): BelongsToMany {
        return $this
            ->belongsToMany(Product::class)
            ->withTimestamps();
    }
}
```

Naming Convention — belongsToMany()

- For the intermediate table name:
 - Eloquent will join the **two related model names in alphabetical order** separated by underscore **_**.
- For the foreign keys name in intermediate table:
 - It is same as hasMany() relationship naming convention.
- But we can specify the intermediate table name and foreign keys name by passing additional arguments to belongsToMany(), e.g., for Product model

```
return $this->belongsToMany(Shop::class,  
    'product_shop',          'product_id',          'shop_id');  
// intermediate_table_name, this side foreign key, other side foreign key
```

Querying Relations

Getting query object for relation

- When we want to get the query object of relation just call **the relationship with method** accessing, e.g. from the previous relationship of `Product` model and `Category` model

```
$category = Category::where('code', 'CT001')->firstOrFail();  
$query = $category->products();  
$products = $query->where('price', '>', 100)->get();
```

- This will generate SQL is like, assume category CT001 has `id = 1`:

```
SELECT * FROM `products` WHERE `category_id` = 1 AND `price` > 100;
```

- Keep in your mind that the **logical grouping is required** when you query with `orWhere()` on relation query.

Aggregating Related Models

- Counting Related Models
 - Sometimes we may want to count the number of related models for a given relationship **without actually loading** the models.
 - To accomplish this, we may use the `withCount()` method.
 - The `withCount()` method which will place a **`{relation_name}_count`** attribute on the resulting models.

```
$categories = Category::withCount('products')->get();
```

```
foreach($categories as $category) {  
    echo $category->name.' has '.$category->products_count;  
}
```

Aggregating Related Models (cont.)

- Deferred Count Loading

- Using the `loadCount()` method, we may load a relationship count **after the parent model has already been retrieved**:

```
$category = Category::where('code', 'CT001')->firstOrFail();  
$category->loadCount('products');
```

```
echo $category->name.' has '.$category->products_count;
```

Aggregating Related Models (cont.)

- Other Aggregate Functions

- In addition to the `withCount()` method, Eloquent provides `withMin()`, `withMax()`, `withAvg()`, and `withSum()` methods.
- These methods will place a `{relation_name}_{function}_{column}` attribute on our resulting models:

```
$categories = Category::withSum('products', 'price')->get();
```

```
foreach($categories as $category) {  
    echo $category->name.' has total value '.$category->products_sum_price;  
}
```

- Like the `loadCount()` method, **deferred versions of these methods are also available.**

N+1 query problem

- N+1 query problem is a common problem in ORM.
 - First, ORM queries to retrieve a **collection of parent objects**, e.g., list.
 - Then, for **each of these N parent objects**, ORM executes a **separated additional query** to **fetch its associated child** (or related) data.
 - This can lead to **significant performance degradation**, especially when dealing with **a large number of records**.
- The primary solution involves using **eager loading** techniques provided by many ORMs to **fetch related data in a single query**.

N+1 query problem (cont.)

- Eloquent provides ***eager loading*** by using method `with()` and specify **relationship name** to be loaded.

```
// assume Product has a relation belongTo to Category.  
$query = Product::orderBy('code')->with('category');
```

- When we call a **finishing query method**, Eloquent also add additional SQL for specified relationships to **retrieve them** in the **same query as collection retrieval**.

Note of the next chapter

- In this chapter, we just define the relationship and query with relationship constrains.
- In the next chapter, we will manipulate the relationship data.
- So, in the example and assignments, you have to add relationships information directly to database.

Example — relation between Product and Shop

- Reuse your previous project from last week, week-08.
- Product model and Shop model have *many-to-many* relationship.
 - From the given file, copy:
 - `2026_08_18_082420_create_product_shop_table.php` to `database/migrations`
 - `ProductShopSeeder.php` to `database/seeder`s
- Migrate and seed by using the following command:

```
php artisan migrate
php artisan db:seed --class=ProductShopSeeder
```

Example — wanted `products.list`

DB - Product List

Ask Gemini

localhost:8000/products

Product

Shop

Product: List

Search

Term

Min Price

Max Price

Filter

Clear

Create Product

< 1 2 >

List of Products

Code	Name	Price	No. of Shops
PD001	Programming PHP	345.00	2
PD002	JavaScript: The Definitive Guide	250.00	3
PD003	Learning PHP, MySQL & JavaScript	450.00	2
PD004	Python Crash Course, 2nd Edition	560.00	1
PD101	Modern PHP Web Development	758.55	4

© Copyright Pachara's Database.

Example — Add relationship

1. Add the following relationship to Product model.

```
// import Illuminate\Database\Eloquent\Relations\BelongsToMany
function shops(): BelongsToMany {
    return $this->belongsToMany(Shop::class)->withTimestamps();
}
```

2. Add the following relationship to Shop model.

```
// import Illuminate\Database\Eloquent\Relations\BelongsToMany
function products(): BelongsToMany {
    return $this->belongsToMany(Product::class)->withTimestamps();
}
```

Example — Add aggregate `withCount()`

3. Now for route `products.list`, we want to show the number of shops that has respective product so:

- In `ProductController` method `list()` change:

```
$query = $this->search($criteria);
```

- to

```
$query = $this->search($criteria)->withCount('shops');
```

Example — update products/list.blade.php

4. Add the column that shows the number of shops in products/list.blade.php.

```
{{-- ... Other table elements --}}

@foreach($products as $product)
    <tr>
        {{-- ... Other table elements --}}
        {{-- Add the following <td> --}}
        <td>{{ $product->shops_count }}</td>
    </tr>
@endforeach
```

Example — wanted products.view and products.view-shops

DB - Product: PD101

localhost:8000/products/PD101

Product Shop

Product: PD101

[View Shops](#) [Update](#) [Delete](#)

Code :: PD101
Name :: Modern PHP Web Development
Price :: 758.55

This comprehensive guide teaches you how to build real-world web applications using modern PHP. You will learn core language features, database integration with MySQL, secure user authentication, and best practices for deploying robust web services.

- Master PHP syntax, object-oriented programming, and MVC architecture.
- Database & Security: Connect safely with PDO, prevent SQL injection, and hash passwords securely.
- Project Workflow: Build a complete dynamic web application step-by-step.

© Copyright Pachara's Database.

DB - Product: PD101: Shops

localhost:8000/products/PD101/shops

Product Shop

Product: PD101: Shops

Search
Term

[Filter](#) [Clear](#)

[< Back](#)

List of Shops for Modern PHP Web Development

Code	Name	Owner	No. of Products
SH001	Pachara Shop	Pachara	2
SH002	Phasit Shop	Phasit	3
SH003	Other Shop	Pachara	3
SH102	MadA Shop	Adam	1

© Copyright Pachara's Database.

Example — update SearchableController

5. We add `Relation` to union type of `$query` to allow passing relation from model.

- **Change:**

```
function filter(Builder $query, array $criteria, array $filterOptions) : void {  
    // method body  
}
```

- **To:**

```
// import Illuminate\Database\Eloquent\Relations\Relation  
function filter(  
    Builder|Relation $query,  
    array $criteria,  
    array $filterOptions,  
) : void {  
    // method body  
}
```

Example — update SearchableController

- update `filterByTerm()` method to match the new signature.

- **Change:**

```
function filterByTerm(  
    Builder $query,  
    string $term,  
    array $termOptions,  
) : void {  
    // method body  
}
```

- **To:**

```
function filterByTerm(  
    Builder|Relation $query,  
    string $term,  
    array $termOptions,  
) : void {  
    // method body  
}
```

Example — update ProductController

6. And update ProductController.

- **Change:**

```
function filter(Builder $query, array $search , array $filterOptions) : void {  
    // method body  
}
```

- **To:**

```
// import Illuminate\Database\Eloquent\Relations\Relation  
function filter(  
    Builder|Relation $query,  
    array $search,  
    array $filterOptions,  
) : void {  
    // method body  
}
```

Example — update ProductController

- update `filterByMinPrice()` and `filterByMaxPrice()` to match the new signature.

```
function filterByMinPrice(  
    Builder $query,  
    float $minPrice,  
) : void {  
    // method body  
}
```

- To:

```
function filterByMinPrice(  
    Builder|Relation $query,  
    float $minPrice,  
) : void {  
    // method body  
}
```

```
function filterByMaxPrice(  
    Builder $query,  
    float $maxPrice,  
) : void {  
    // method body  
}
```

- To:

```
function filterByMaxPrice(  
    Builder|Relation $query,  
    float $maxPrice,  
) : void {  
    // method body  
}
```

Example — method for getting shops of product

7. Add the following method in ProductController.

```
function viewShops(string $productCode, ServerRequestInterface $request): View
{
    $product = $this->find($productCode);
    $shopController = \resolve(ShopController::class);
    $criteria = $shopController->prepareCriteria($request->getQueryParams());
    $filterOptions = $shopController->getFilterOptions();
    $query = $product->shops()->withCount('products');
    $shopController->filter($query, $criteria, $filterOptions);

    return view('products.view-shops', [
        'criteria' => $criteria,
        'product' => $product,
        'shops' => $query->paginate($shopController::MAX_ITEMS),
    ]);
}
```

Example — create products/view-shops.blade.php

8. This template is very similar to shops/list.blade.php.

- Change template in `@extends` to `'products.main'`.
 - Title is `"{$product->code}: Shops"`
- Change search and clear URL to route `products.view-shops` in **section header**.

```
<search><form action="{{ route('products.view-shops', [
    'product' => $product->code,
1) }}" method="get" class="app-cmp-search-form">
    {{-- ... Other form controls --}}
    <a href="{{ route('products.view-shops', [
        'product' => $product->code,
1) }}">Clear</a>
</form></search>
```

Example — create products/view-shops.blade.php (cont.)

- Add back link to products.view.

```
<nav class="app-cmp-links-bar">
  <ul>
    {{-- Change show create-form to back link to product-view --}}
    <li>
      <a href="{{ route('products.view', [
        'product' => $product->code,
      ]) }}">&lt; Back</a>
    </li>
  </ul>

  {{ $shops->withQueryString()->links() }}
</nav>
```

Example — create products/view-shops.blade.php (cont.)

- Update table **caption** to use **`$product->name`**.

```
<caption>List of Shops for {{ $product->name }}</caption>
```

- Add **No. of Products** column.

```
{{-- ... Other table elements --}}
```

```
@foreach($shops as $shop)
```

```
    <tr>
```

```
        {{-- ... Other table elements --}}
```

```
        {{-- Add the following <td> --}}
```

```
        <td>{{ $shop->products_count }}</td>
```

```
    </tr>
```

```
@endforeach
```

Example — add route to routes/web.php

9. Add route `product-view-shop` in `routes/web.php`.

```
// In Product controller group.  
// After route products.delete.  
Route::get(  
    '/shops',  
    'viewShops',  
)->name('view-shops');
```

Example — update products/view.blade.php

10. Update products/view.blade.php to have link to route products.view-shops.

```
{{-- Add Show Shop to navigation --}}  
<nav>  
  <ul>  
    <li>  
      <a href="{{ route('products.view-shops', [  
        'product' => $product->code,  
      ] ) }}">View Shops</a>  
    </li>  
    {{-- ... link to Update --}}  
    {{-- ... link to Delete --}}  
  </ul>  
</nav>
```

References

- <https://laravel.com/docs/13.x/eloquent-relationships>